IoT FM transmission silence detection

Breadboard prototype • ESP32 DevKit V1 • ADS1115 • LM358 • BAT43 • RDA5807M

Purpose: receive an FM station, convert its audio peaks into a slowly changing voltage, and send that measurement over Wi-Fi to a Windows PC dashboard. Silence means low audio for a configured duration; this does not prove transmitter or RF carrier failure.

Receiver pinout pending: your receiver has not arrived and its board model is unknown. This guide assigns connections by pin **label/function**, not by physical position. Verify the module datasheet before wiring. A bare receiver IC needs its reference circuit, crystal, RF components and a suitable PCB; do not put it directly into this breakout-board layout.

1. Additional parts required

Part	Quantity / use
Resistors	30 kΩ x1; 10 kΩ x2; 100 kΩ x2; 1 kΩ x2. Use 1% if available.
Capacitors	1 µF nonpolar/film x2 (audio coupling and envelope); 100 nF ceramic x3 (supply bypass); 10 µF x1 (reference filter, positive toward divider). Voltage rating 6.3 V or higher.
Other	Breadboard, jumpers, USB data cable, multimeter, and antenna suitable for your receiver module.
Main parts	One ESP32 DevKit V1, LM358 DIP-8 (or correctly mapped adapter), one BAT43 diode, one ADS1115 breakout, one documented RDA5807M breakout.
I²C pull-ups	Only if not fitted: 4.7 kΩ SDA to 3.3 V and 4.7 kΩ SCL to 3.3 V. Do not add blindly if both modules already have pull-ups.

Power: USB powers ESP32. Use ESP32 **3V3** for this entire analog/I²C circuit; join all grounds. Do not connect USB/VIN/5V to the 3.3 V rail. Never apply 5 V logic to ESP32 or this ADS1115. Power off while changing wiring. Breadboard power rails may be split; check continuity.

Design limit: the LM358 is not rail-to-rail. This circuit biases audio to about 0.825 V and is intended for audio peaks no larger than about ±0.4 V at LM358 pin 3 (roughly 0.425–1.225 V). Meter-check the bias and use an oscilloscope for peak/clipping checks if available. Large headphone/speaker signals are unsuitable without attenuation. Do not assume the listed parts alone make a calibrated instrument.

2. Digital bus and power connections

From	To / purpose
ESP32 3V3	ADS1115 VDD/VCC; LM358 pin 8; RDA5807M breakout 3.3 V supply only if its documentation permits.
ESP32 GND	ADS1115 GND; LM358 pin 4; receiver GND; analog ground rail.
ESP32 GPIO21	ADS1115 SDA and receiver SDA / SDIO.
ESP32 GPIO22	ADS1115 SCL and receiver SCL / SCLK.
ADS1115 ADDR	GND: I²C address 0x48.
ADS1115 A0	Envelope E through 1 kΩ (R7). Do not use digital GPIO as the audio input.
ADS1115 A1/A2/A3, ALERT	Leave disconnected; firmware reads only A0.
Receiver LOUT / L audio	Only to C1 audio coupling capacitor below. Leave ROUT disconnected for this single-channel prototype; never short LOUT and ROUT together.
Receiver ANT	Antenna according to the actual module instructions. Some boards use a headphone lead as the antenna.
Decoupling	100 nF between supply and ground near each of LM358, ADC and receiver. Keep analog leads short.

3. LM358 orientation — DIP-8, top view

Place the DIP IC across the breadboard centre gap. With the notch at the top, pin 1 is upper left. Down the left side: **1 OUT A, 2 IN− A, 3 IN+ A, 4 GND**. Up the right side: **5 IN+ B, 6 IN− B, 7 OUT B, 8 V+**. Pins 1 and 8 are opposite each other at the notch end. Confirm the manufacturer datasheet; other packages require an adapter and may differ.

4. Build the 0.825 V reference first

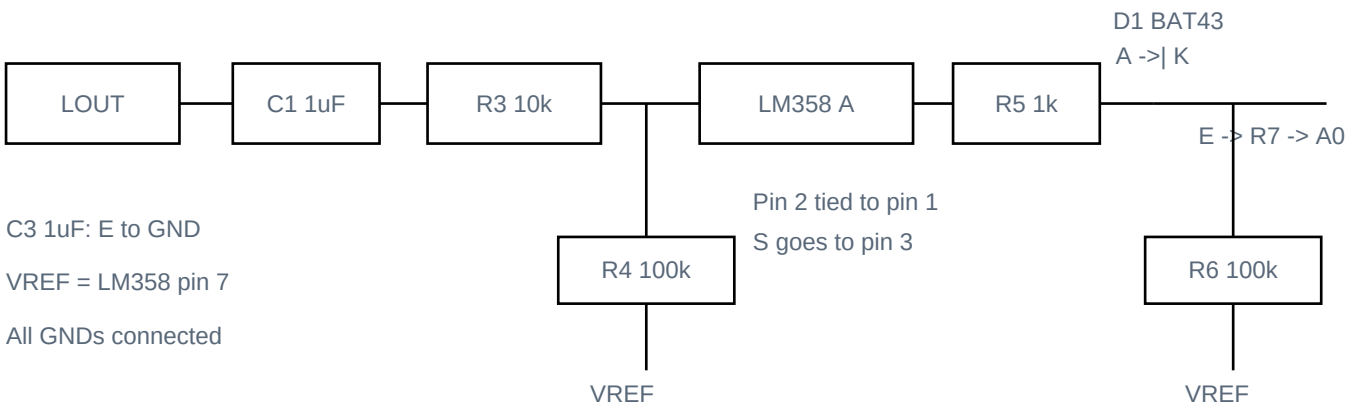
3V3 → **R1 30 kΩ** → divider node B → **R2 10 kΩ** → GND. Put C2 10 µF from B to GND (+ to B). Connect B to LM358 pin 5. Connect LM358 pin 6 directly to pin 7. Pin 7 is **VREF**, approximately 3.3 × 10/(30+10) = 0.825 V. This uses amplifier B as a reference buffer.

Check with a multimeter: pin 8 about 3.3 V; pin 4 ground; B and pin 7 about 0.825 V. Stop if these are incorrect. VREF is an analog bias, **not a ground rail**.

5. Audio and envelope — exact net connections

Connection	Part / instruction
Receiver LOUT → C1 → R3 → S	C1 = 1 µF nonpolar coupling capacitor; R3 = 10 kΩ. S is the biased audio node.
S → VREF	R4 = 100 kΩ. This sets DC bias at the input.
S → LM358 pin 3	Amplifier A non-inverting input.
LM358 pin 2 → pin 1	Direct feedback; amplifier A is a unity-gain buffer.
Pin 1 → R5 → D1 anode	R5 = 1 kΩ limits charging current and isolates the capacitor load.
D1 cathode → E	D1 = BAT43. The cathode is the banded end. E is the envelope node.
E → VREF	R6 = 100 kΩ; envelope discharge path.
E → GND	C3 = 1 µF nonpolar. Nominal decay time R6 × C3 = 0.1 s.
E → R7 → ADS1115 A0	R7 = 1 kΩ. ADC and amplifier share 3.3 V and GND.

With quiet input, E settles near VREF because R6 pulls it there. Audio peaks above VREF plus the diode drop charge C3. The ADC measures E, and the dashboard subtracts the measured quiet baseline. The BAT43 forward voltage varies with current and temperature; weak audio can fall below its useful detection range.



Analog verification: before connecting ADS1115 A0, check E is near 0.825 V at quiet and remains between GND and 3.3 V. The ADC gain setting ±4.096 V is a conversion range, not permission to exceed its 3.3 V supply. The 1 kΩ resistor is not an overvoltage clamp.

6. Bring-up before the receiver arrives

1. Build only the reference circuit and check its voltages. Then build the buffer/envelope. With the receiver absent, connect the left/input side of C1 to GND. S, buffer pin 1 and E should settle near VREF. Do not ground S, VREF or a powered receiver output.

2. Set **USE_RADIO 0** in the firmware for receiver-free ADC testing. Upload using the supplied README. Check readings near the measured envelope baseline. A stable quiet reading verifies bias and telemetry, not audio sensitivity.

3. For a controlled audio test, disconnect the grounding jumper and apply an isolated/low-voltage signal-generator sine wave: about 1 kHz, 0.8 V peak-to-peak, zero DC offset, to the input side of C1; generator GND to common GND. Check S stays in its permitted range. E should rise by a measurable amount. Start at lower amplitude and increase gradually. Never use mains or an unknown speaker output.

4. If you have no suitable audio test source, use dashboard simulation now and defer analog sound validation until the documented receiver arrives. Do not claim a silent baseline proves sound detection.

7. Windows dashboard and ESP32 connection

Extract the ZIP and install Python 3.10+. First run **py -3 server.py --create-user "+YOUR_NUMBER"** with your international phone number, then choose a password (12+ characters). Run **py -3 server.py --demo**; open **http://localhost:8000** and sign in. Simulation cycles normal, low audio, carrier/no audio and transmitter off. Stop with Ctrl+C before hardware mode.

For hardware: run **py -3 server.py --host 0.0.0.0**. Put the PC Wi-Fi IPv4 address from **ipconfig** into the ESP32 SERVER_URL. Both devices must be on the same trusted LAN; ESP32 needs 2.4 GHz Wi-Fi. Permit Python on Windows Firewall Private networks only. Generate a device token using **py -3 server.py --create-device-token** and copy it into local firmware. No port forwarding: HTTP is unencrypted even with login. See README.md and HARDWARE_API.md for current setup and telemetry fields.

In Arduino IDE install the Espressif ESP32 board package, Adafruit ADS1X15 and PU2CLR RDA5807 libraries. Choose DOIT ESP32 DEVKIT V1. Set Wi-Fi, PC address and local FM frequency in the sketch. Serial Monitor is 115200 baud. The full README gives upload and troubleshooting steps.

8. Calibration and limitations

Fix the radio volume. With a known silent input, wait 2 s and record/use the latest reading as the quiet baseline. Restore normal audio and choose a threshold above quiet noise but below measured programme levels. Defaults are silence below 45 mV rise, normal at or above 100 mV, and 5 s silence delay; calibrate all three. Low audio falls between the thresholds. Verify recovery and device offline within 3 s after telemetry stops.

Prototype limitations: one audio channel; diode dead zone; receiver hiss may mask loss of station; carrier inference requires RSSI calibration; confirmed transmitter off needs independent hardware status; settings/history are not persistent. If quiet and programme levels overlap, this circuit is unsuitable at that level and needs amplification or a precision detector. Firmware and circuit are supplied for bench validation; Arduino compilation, actual receiver pin order and physical operation have not been verified here.